UTS

PEMBELAJARAN MESIN



NAMA : TAQIYA RIZKIPUDENTA

NIM : 202431180

KELAS : G

DOSEN : DR. DRA. DWINA KUSWARDANI, M.KOM

NO.PC :

ASISTEN : 1. MORGAN IMMANUEL NAINGGOLAN

2. MUHAMMAD ICHWAN FAUZI

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

DAFTAR ISI 2
BAB I 4
PENDAHULUAN 4
1.1 Rumusan Masalah 4
1.2 Tujuan Masalah 4
1.3 Manfaat Masalah 4
BAB II 5
LANDASAN TEORI 5
2.1 Pembelajaran Mesin (Machine Learning) 5
2.2 Klasifikasi (Classification) 5
2.3 Dataset 6
2.4 Exploratory Data Analysis (EDA) 7
2.5 Data Preprocessing 7
2.5.1 Pemeriksaan Nilai yang Hilang (Missing Values) 7
2.5.2 Penghapusan Data Duplikat 8
2.5.3 Encoding Data Kategorikal 8
2.5.4 Feature Scaling 8
2.6 Pembagian Data (Train-Test Split) 9
2.7 Algoritma Naive Bayes 9
2.8 Algoritma K-Nearest Neighbors (KNN) 10
2.9 Evaluasi Model Klasifikasi 11
2.9.1 Confusion Matrix 11
2.9.2 Accuracy 11
2.9.3 Precision 11
2.9.4 Recall 12
2.9.5 F1-Score 12
BAB III 13
HASIL 13
3.1 Import Library dan Membaca Dataset 13
3.2 Exploratory Data Analysis (EDA) 13
3.3 Preprocessing Data 15
3.4 Implementasi Algoritma Naive Bayes 16
3.5 Implementasi Algoritma K-Nearest Neighbor (KNN) 17

3.6 Evaluasi Model & Visualisasi Perbandingan Model 18
3.7 Confusion Matrix 19
3.8 Menampilkan Hasil Prediksi Aktual dan Model 20
3.9 Kesimpulan Hasil 21
BAB IV 22
PENUTUP 22
DAFTAR PUSTAKA 23

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana karakteristik dataset Counter-Strike 2 yang digunakan dalam proses klasifikasi?
2. Bagaimana melakukan preprocessing pada dataset Counter-Strike 2 agar dapat digunakan dalam pemodelan machine learning?
3. Bagaimana penerapan algoritma Naive Bayes dan K-Nearest Neighbors (KNN) dalam memprediksi tim pemenang ronde?
4. Bagaimana perbandingan kinerja algoritma Naive Bayes dan KNN berdasarkan hasil evaluasi model?

1.2 Tujuan Masalah

1. Mengetahui karakteristik dataset Counter-Strike 2 melalui proses Exploratory Data Analysis (EDA).
2. Melakukan preprocessing data agar dataset siap digunakan dalam proses klasifikasi.
3. Menerapkan algoritma Naive Bayes dan K-Nearest Neighbors (KNN) untuk memprediksi tim pemenang ronde.
4. Mengetahui tingkat akurasi dan kinerja masing-masing algoritma dalam melakukan klasifikasi.
5. Membandingkan hasil klasifikasi algoritma Naive Bayes dan KNN untuk mengetahui algoritma dengan performa terbaik pada dataset yang digunakan.

1.3 Manfaat Masalah

1. Menambah pemahaman mengenai penerapan machine learning dalam menyelesaikan permasalahan klasifikasi.
2. Melatih kemampuan dalam melakukan eksplorasi, analisis, dan preprocessing data sebelum proses pemodelan.
3. Memahami cara kerja algoritma Naive Bayes dan K-Nearest Neighbors (KNN) dalam melakukan prediksi.
4. Mengetahui dan membandingkan kinerja algoritma berdasarkan hasil evaluasi model.
5. Menjadi referensi dalam penerapan metode klasifikasi pada permasalahan prediksi berbasis data.

BAB II
LANDASAN TEORI

2.1 Pembelajaran Mesin (Machine Learning)

Pembelajaran mesin (machine learning) merupakan bagian dari kecerdasan buatan yang memungkinkan komputer mempelajari pola dari data untuk menghasilkan prediksi atau keputusan. Proses pembelajaran dilakukan dengan menggunakan data sebagai dasar pembentukan model. Model yang telah dilatih dapat digunakan untuk mengenali pola dan memprediksi hasil berdasarkan data baru.

Pembelajaran mesin dapat digunakan untuk menyelesaikan berbagai permasalahan, salah satunya adalah klasifikasi. Klasifikasi merupakan proses mengelompokkan data ke dalam kategori tertentu berdasarkan karakteristik yang dimiliki. Penerapan pembelajaran mesin dalam klasifikasi dapat ditemukan pada berbagai bidang, seperti kesehatan, pendidikan, keuangan, dan analisis data.

Menurut Loka dan Marsal (2023), algoritma K-Nearest Neighbor dan Naive Bayes dapat digunakan untuk melakukan klasifikasi terhadap data status gizi balita. Penelitian tersebut membandingkan performa kedua algoritma berdasarkan hasil klasifikasi yang diperoleh. Hal ini menunjukkan bahwa algoritma pembelajaran mesin dapat diterapkan pada berbagai jenis permasalahan klasifikasi dengan menyesuaikan karakteristik dataset yang digunakan.

Pada praktikum ini, pembelajaran mesin diterapkan untuk memprediksi tim yang memenangkan suatu ronde dalam permainan Counter-Strike 2. Model mempelajari pola berdasarkan kondisi permainan, seperti waktu tersisa, skor, kesehatan, armor, jumlah uang, dan status penanaman bom.

2.2 Klasifikasi (Classification)

Klasifikasi merupakan salah satu metode dalam pembelajaran mesin yang bertujuan untuk menentukan kelas atau kategori dari suatu data. Proses klasifikasi dilakukan dengan mempelajari hubungan antara atribut atau fitur dengan label kelas yang telah diketahui.

Dalam klasifikasi, data umumnya dibagi menjadi data latih (training data) dan data uji (testing data). Data latih digunakan untuk membangun model, sedangkan data uji digunakan untuk mengevaluasi kemampuan model dalam memprediksi data yang belum pernah digunakan selama proses pelatihan.

Penelitian Prayogo, Fauziah, dan Winarsih (2023) menerapkan algoritma Naive Bayes dan K-Nearest Neighbor untuk melakukan klasifikasi judul artikel jurnal ilmiah. Penelitian tersebut menunjukkan bahwa kedua algoritma dapat digunakan untuk menyelesaikan permasalahan klasifikasi, meskipun hasil performanya dapat berbeda berdasarkan metode dan karakteristik data yang digunakan.

Pada praktikum ini, klasifikasi dilakukan untuk menentukan kelas round_winner, yaitu tim Counter-Terrorist (CT) atau Terrorist (T). Atribut kondisi permainan digunakan sebagai fitur masukan, sedangkan round_winner menjadi variabel target yang diprediksi oleh model.

2.3 Dataset

Dataset merupakan kumpulan data yang digunakan sebagai dasar dalam proses analisis dan pemodelan pembelajaran mesin. Dataset umumnya terdiri dari baris yang merepresentasikan observasi dan kolom yang merepresentasikan atribut atau variabel dari setiap observasi.

Dataset yang digunakan dalam praktikum ini adalah dataset Counter-Strike 2 yang berisi informasi mengenai kondisi permainan pada setiap ronde. Dataset tersebut terdiri dari 2.000 baris data dan 14 kolom, yang mencakup atribut kondisi permainan serta variabel target berupa hasil kemenangan ronde.

Atribut yang terdapat pada dataset antara lain:

Atribut	**Keterangan**
time_left	Waktu yang tersisa dalam ronde
ct_score	Skor tim Counter-Terrorist
t_score	Skor tim Terrorist
map	Peta yang digunakan dalam permainan
bomb_planted	Status apakah bom telah ditanam
ct_health	Total kondisi kesehatan tim CT
t_health	Total kondisi kesehatan tim T
ct_armor	Total armor tim CT
t_armor	Total armor tim T
ct_money	Total uang tim CT
t_money	Total uang tim T
ct_helmets	Jumlah helm yang dimiliki tim CT
t_helmets	Jumlah helm yang dimiliki tim T
round_winner	Tim yang memenangkan ronde

Atribut-atribut tersebut digunakan untuk mempelajari hubungan antara kondisi permainan dengan hasil akhir ronde. Dengan memanfaatkan informasi tersebut, model diharapkan dapat mengenali pola yang berkaitan dengan kemenangan tim CT atau T.

2.4 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) merupakan proses eksplorasi data yang dilakukan untuk memahami karakteristik, struktur, distribusi, dan pola yang terdapat dalam dataset sebelum dilakukan pemodelan. EDA membantu mengidentifikasi kondisi data sehingga proses preprocessing dan pemilihan algoritma dapat dilakukan dengan lebih tepat.

EDA dapat dilakukan melalui pemeriksaan ukuran dataset, tipe data, nilai yang hilang, data duplikat, distribusi kelas, serta statistik deskriptif. Proses ini juga dapat menggunakan visualisasi seperti histogram, diagram batang, dan diagram distribusi untuk membantu memahami pola data.

Dalam penelitian Loka dan Marsal (2023), data status gizi balita digunakan untuk membandingkan kinerja algoritma KNN dan Naive Bayes. Pemahaman terhadap karakteristik data menjadi bagian penting agar proses klasifikasi dapat dilakukan sesuai dengan kebutuhan penelitian.

Pada praktikum ini, EDA dilakukan untuk mengetahui karakteristik dataset Counter-Strike 2, termasuk distribusi kelas round_winner, persebaran atribut, serta kondisi data yang akan digunakan dalam proses klasifikasi.

2.5 Data Preprocessing

Data preprocessing merupakan tahapan pengolahan data sebelum digunakan dalam proses pelatihan model pembelajaran mesin. Tahapan ini bertujuan untuk memastikan bahwa data memiliki format dan kualitas yang sesuai dengan kebutuhan algoritma.

Data yang diperoleh dari suatu sumber dapat memiliki berbagai permasalahan, seperti nilai yang hilang, data duplikat, perbedaan tipe data, serta skala nilai yang berbeda. Oleh karena itu, preprocessing diperlukan agar proses pemodelan dapat berjalan dengan baik.

Dalam penelitian mengenai klasifikasi, pemilihan tahapan preprocessing perlu disesuaikan dengan karakteristik data dan algoritma yang digunakan. Krisna dan Salamah (2022) membandingkan algoritma Naive Bayes dan KNN dalam klasifikasi berita hoaks kesehatan. Penelitian tersebut memperlihatkan bahwa kedua algoritma dapat digunakan pada permasalahan klasifikasi, dengan hasil performa yang berbeda.

Pada praktikum ini, preprocessing dilakukan melalui beberapa tahapan berikut.

2.5.1 Pemeriksaan Nilai yang Hilang (Missing Values)

Nilai yang hilang atau missing values merupakan kondisi ketika suatu atribut tidak memiliki nilai pada sebagian observasi. Keberadaan nilai yang hilang dapat memengaruhi proses analisis dan pemodelan, terutama apabila algoritma tidak dapat memproses nilai tersebut secara langsung.

Pemeriksaan missing values dilakukan untuk mengetahui apakah terdapat atribut yang memiliki nilai kosong. Apabila ditemukan nilai yang hilang, penanganannya dapat

dilakukan melalui penghapusan data atau pengisian nilai berdasarkan metode tertentu, sesuai dengan karakteristik data.

Pada dataset Counter-Strike 2 yang digunakan dalam praktikum ini, hasil pemeriksaan menunjukkan bahwa tidak terdapat nilai yang hilang pada seluruh kolom.

2.5.2 Penghapusan Data Duplikat
Data duplikat merupakan data yang memiliki kesamaan dengan baris data lainnya. Keberadaan data duplikat dapat menyebabkan suatu observasi memperoleh bobot lebih besar dalam proses pembelajaran model karena data yang sama muncul lebih dari satu kali.

Penghapusan data duplikat dilakukan untuk mengurangi pengulangan observasi yang sama. Pada dataset yang digunakan, ditemukan 39 baris data duplikat dari total 2.000 baris. Dengan menghapus seluruh baris duplikat, jumlah data menjadi 1.961 baris.

2.5.3 Encoding Data Kategorikal
Encoding merupakan proses mengubah data kategorikal menjadi bentuk numerik agar dapat diproses oleh algoritma pembelajaran mesin yang membutuhkan masukan numerik.

Salah satu metode encoding adalah One-Hot Encoding. Metode ini mengubah setiap kategori menjadi kolom biner yang menunjukkan keberadaan kategori tersebut. One-Hot Encoding sesuai digunakan pada variabel kategorikal yang tidak memiliki urutan tertentu.

Pada dataset Counter-Strike 2, atribut map merupakan data kategorikal yang berisi nama peta permainan. Oleh karena itu, atribut tersebut perlu diubah ke bentuk numerik sebelum digunakan dalam pemodelan.

2.5.4 Feature Scaling
Feature scaling merupakan proses menyamakan skala nilai pada fitur numerik agar perbedaan rentang nilai tidak menyebabkan suatu fitur mendominasi proses perhitungan.

Salah satu metode yang umum digunakan adalah standardization atau standardisasi. Standardisasi mengubah nilai fitur berdasarkan rata-rata dan standar deviasi data.

Rumus standardisasi adalah:

$$ z=\frac{x-\mu}{\sigma} $$

Keterangan:

- z = nilai setelah standardisasi
- x = nilai awal
- μ = rata-rata data
- σ = standar deviasi

Standardisasi penting terutama pada algoritma KNN karena algoritma tersebut menentukan kedekatan antarobservasi melalui perhitungan jarak. Fitur dengan rentang nilai besar dapat memengaruhi jarak secara dominan apabila tidak dilakukan penyamaan skala.

2.6 Pembagian Data (Train-Test Split)

Pembagian data merupakan proses memisahkan dataset menjadi data latih (training data) dan data uji (testing data). Data latih digunakan untuk membangun model, sedangkan data uji digunakan untuk mengukur kemampuan model dalam memprediksi data yang belum pernah digunakan pada proses pelatihan.

Pembagian data diperlukan agar evaluasi model tidak hanya didasarkan pada data yang telah dipelajari. Dengan demikian, hasil evaluasi dapat memberikan gambaran mengenai kemampuan model dalam melakukan klasifikasi terhadap data baru.

Pada praktikum ini, dataset dibagi menggunakan perbandingan 80% data latih dan 20% data uji. Pembagian dilakukan secara stratified agar proporsi kelas pada data latih dan data uji tetap mendekati distribusi kelas pada dataset.

2.7 Algoritma Naive Bayes

Naive Bayes merupakan algoritma klasifikasi berbasis probabilitas yang menggunakan Teorema Bayes untuk menentukan kemungkinan suatu data termasuk ke dalam kelas tertentu. Algoritma ini memiliki asumsi bahwa setiap fitur bersifat independen terhadap fitur lainnya apabila kelasnya telah diketahui.

Naive Bayes dapat digunakan untuk memprediksi kelas berdasarkan probabilitas fitur yang dimiliki oleh suatu observasi. Kelas dengan nilai probabilitas posterior tertinggi akan dipilih sebagai hasil prediksi.

Teorema Bayes dirumuskan sebagai berikut:

\[P(C\mid X)=\frac{P(X\mid C)P(C)}{P(X)} \]

Keterangan:

- \(P(C\mid X)\) = probabilitas kelas \(C\) berdasarkan fitur \(X\)
- \(P(X\mid C)\) = probabilitas fitur \(X\) jika kelas \(C\) diketahui
- \(P(C)\) = probabilitas awal kelas
- \(P(X)\) = probabilitas fitur \(X\)

Dalam klasifikasi, Naive Bayes menentukan kelas berdasarkan probabilitas posterior dari setiap kelas. Asumsi independensi antarfitur membuat algoritma ini relatif sederhana dan efisien untuk diterapkan.

Krisna dan Salamah (2022) menerapkan Naive Bayes dan KNN untuk mengklasifikasikan berita hoaks kesehatan di media sosial Twitter. Penelitian tersebut membandingkan performa kedua algoritma berdasarkan nilai akurasi yang diperoleh.

Hasil penelitian menunjukkan bahwa algoritma KNN memperoleh akurasi sebesar 68%, sedangkan Naive Bayes memperoleh akurasi sebesar 66%. Hasil tersebut menunjukkan bahwa performa algoritma dapat berbeda bergantung pada karakteristik data yang digunakan.

Pada praktikum ini, algoritma Naive Bayes digunakan untuk memprediksi round_winner berdasarkan fitur-fitur kondisi permainan Counter-Strike 2. Jenis Naive Bayes yang digunakan adalah Gaussian Naive Bayes, yang sesuai untuk fitur numerik dengan asumsi distribusi normal pada setiap fitur berdasarkan kelasnya.

2.8 Algoritma K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) merupakan algoritma klasifikasi yang menentukan kelas suatu data berdasarkan kelas dari sejumlah tetangga terdekat pada data latih. KNN termasuk metode instance-based learning atau lazy learning, karena proses pembelajaran utamanya dilakukan dengan menyimpan data latih dan menentukan tetangga terdekat ketika prediksi dilakukan.

Pada proses klasifikasi, KNN menghitung jarak antara data yang akan diprediksi dengan data latih. Selanjutnya, algoritma memilih sejumlah \(k\) tetangga dengan jarak terkecil. Kelas yang paling banyak muncul di antara tetangga tersebut menjadi hasil prediksi.

Loka dan Marsal (2023) menerapkan algoritma KNN dan Naive Bayes untuk klasifikasi status gizi balita. Penelitian tersebut menunjukkan bahwa KNN dapat digunakan untuk melakukan klasifikasi terhadap data yang memiliki beberapa atribut sebagai fitur masukan.

Salah satu metode pengukuran jarak yang digunakan pada KNN adalah Euclidean Distance. Rumus Euclidean Distance adalah:

\[d(x,y)=\sqrt{\sum_{i=1}^{n}(x_i-y_i)^2} \]

Keterangan:

- \(d(x,y)\) = jarak antara dua data
- \(x_i\) = nilai fitur ke-\(i\) pada data pertama
- \(y_i\) = nilai fitur ke-\(i\) pada data kedua
- \(n\) = jumlah fitur

Nilai \(k\) merupakan jumlah tetangga terdekat yang digunakan untuk menentukan kelas prediksi. Pemilihan nilai \(k\) dapat memengaruhi performa klasifikasi. Nilai \(k\) yang terlalu kecil dapat membuat model sensitif terhadap noise, sedangkan nilai \(k\) yang terlalu besar dapat menyebabkan model kurang mampu menangkap pola lokal pada data.

Pada praktikum ini, KNN digunakan untuk memprediksi tim pemenang ronde berdasarkan kemiripan kondisi permainan dengan data latih. Proses standardisasi

dilakukan agar perhitungan jarak tidak didominasi oleh fitur yang memiliki rentang nilai lebih besar.

2.9 Evaluasi Model Klasifikasi

Evaluasi model merupakan tahapan untuk mengukur kemampuan algoritma dalam melakukan klasifikasi. Evaluasi dilakukan dengan membandingkan hasil prediksi model terhadap label sebenarnya pada data uji.

Salah satu metode evaluasi yang digunakan adalah confusion matrix. Confusion matrix menyajikan perbandingan antara kelas aktual dan kelas hasil prediksi. Berdasarkan confusion matrix, dapat dihitung berbagai metrik evaluasi, seperti accuracy, precision, recall, dan F1-score.

Penelitian Loka dan Marsal (2023) membandingkan algoritma KNN dan Naive Bayes berdasarkan hasil klasifikasi yang diperoleh. Perbandingan tersebut menunjukkan pentingnya evaluasi model untuk mengetahui algoritma yang memiliki performa lebih baik pada dataset tertentu.

2.9.1 Confusion Matrix

Confusion matrix merupakan tabel yang menunjukkan jumlah prediksi benar dan salah dari suatu model klasifikasi. Pada klasifikasi biner, confusion matrix terdiri dari empat komponen, yaitu:

1. True Positive (TP), yaitu data positif yang diprediksi sebagai positif.
2. True Negative (TN), yaitu data negatif yang diprediksi sebagai negatif.
3. False Positive (FP), yaitu data negatif yang diprediksi sebagai positif.
4. False Negative (FN), yaitu data positif yang diprediksi sebagai negatif.

Pada klasifikasi pemenang Counter-Strike 2, kelas dapat ditetapkan sebagai CT dan T. Confusion matrix digunakan untuk mengetahui jumlah prediksi tim pemenang yang benar maupun yang keliru.

2.9.2 Accuracy

Accuracy merupakan metrik evaluasi yang mengukur proporsi jumlah prediksi benar terhadap seluruh data yang dievaluasi.

Rumus accuracy adalah:

$$ Accuracy=\frac{TP+TN}{TP+TN+FP+FN} $$

Nilai accuracy berada pada rentang 0 hingga 1 atau 0% hingga 100%. Semakin tinggi nilai accuracy, semakin besar proporsi prediksi yang sesuai dengan kelas aktual.

2.9.3 Precision

Precision merupakan metrik yang mengukur proporsi prediksi positif yang benar-benar termasuk ke dalam kelas positif.

Rumus precision adalah:

$$ Precision=\frac{TP}{TP+FP} $$

Precision digunakan untuk mengetahui seberapa tepat model ketika memprediksi suatu kelas.

2.9.4 Recall

Recall merupakan metrik yang mengukur kemampuan model dalam menemukan seluruh data yang benar-benar termasuk ke dalam kelas positif.

Rumus recall adalah:

$$ Recall=\frac{TP}{TP+FN} $$

Recall digunakan untuk mengetahui seberapa banyak data dari suatu kelas yang berhasil dikenali oleh model.

2.9.5 F1-Score

F1-score merupakan metrik evaluasi yang menggabungkan precision dan recall melalui rata-rata harmonik. Metrik ini digunakan untuk melihat keseimbangan antara precision dan recall.

Rumus F1-score adalah:

$$ F1=2\times\frac{Precision\times Recall}{Precision+Recall} $$

F1-score dapat memberikan informasi tambahan ketika distribusi kelas tidak sepenuhnya seimbang. Pada praktikum ini, accuracy, precision, recall, F1-score, dan confusion matrix digunakan untuk mengevaluasi serta membandingkan hasil klasifikasi Naive Bayes dan KNN.

BAB III
HASIL

3.1 Import Library dan Membaca Dataset

1. Import library dan membaca dataset

[1]:
```
# Import library
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# Membaca dataset
df = pd.read_csv('03_counter_strike_2.csv')

# Menampilkan 5 data pertama
df.head()

# TAQIYA RIZKIPUDENTA_202431180
```

[1]:

	time_left	ct_score	t_score	map	bomb_planted	ct_health	t_health	ct_armor	t_armor	ct_money	t_money	ct_helmets	t_helmets	round_winner
0	175.00	0.0	0.0	de_dust2	False	500.0	500.0	0.0	0.0	4000.0	4000.0	0.0	0.0	CT
1	156.03	0.0	0.0	de_dust2	False	500.0	500.0	400.0	300.0	600.0	650.0	0.0	0.0	CT
2	96.03	0.0	0.0	de_dust2	False	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0	CT
3	76.03	0.0	0.0	de_dust2	False	391.0	400.0	294.0	200.0	750.0	500.0	0.0	0.0	CT
4	174.97	1.0	0.0	de_dust2	False	500.0	500.0	192.0	0.0	18350.0	10750.0	0.0	0.0	CT

Pada tahap ini, dilakukan import library yang dibutuhkan untuk proses pengolahan data dan klasifikasi, yaitu Pandas, NumPy, Matplotlib, Seaborn, serta beberapa modul dari Scikit-learn. Library tersebut digunakan untuk membaca dataset, melakukan preprocessing, membangun model Naive Bayes dan KNN, serta mengevaluasi hasil prediksi.

Dataset Counter-Strike 2 dibaca menggunakan fungsi pd.read_csv() dengan nama file 03_counter_strike_2.csv. Fungsi df.head() digunakan untuk menampilkan lima data pertama sebagai gambaran awal isi dataset.

Berdasarkan output, dataset memiliki atribut seperti time_left, ct_score, t_score, map, bomb_planted, ct_health, t_health, ct_armor, t_armor, ct_money, t_money, ct_helmets, dan t_helmets. Atribut round_winner digunakan sebagai variabel target untuk menentukan pemenang ronde, yaitu Counter-Terrorist (CT) atau Terrorist (T).

3.2 Exploratory Data Analysis (EDA)

Pada tahap EDA, dilakukan analisis terhadap ukuran, struktur, kelengkapan, dan distribusi data. Berdasarkan output, dataset memiliki 2.000 baris data dan 14 atribut. Seluruh atribut memiliki 2.000 data non-null, sehingga tidak ditemukan missing values pada dataset.

2. Exploratory Data Analysis (EDA)

```
[2]: # Menampilkan ukuran dataset
print("Ukuran dataset:", df.shape)

# Menampilkan informasi dataset
df.info()

# Mengecek missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Mengecek data duplikat
print("\nJumlah data duplikat:", df.duplicated().sum())

# Melihat distribusi target
print("\nDistribusi round_winner:")
print(df['round_winner'].value_counts())

# Visualisasi distribusi target
sns.countplot(data=df, x='round_winner')
plt.title('Distribusi Round Winner')
plt.xlabel('Round Winner')
plt.ylabel('Jumlah Data')
plt.show()

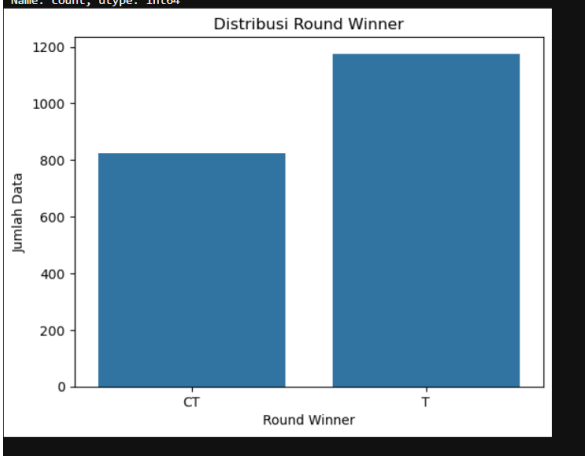
# TAQIYA RIZKIPUDENTA_202431180
```

```
Ukuran dataset: (2000, 14)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 14 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   time_left      2000 non-null   float64
 1   ct_score       2000 non-null   float64
 2   t_score        2000 non-null   float64
 3   map            2000 non-null   object
 4   bomb_planted   2000 non-null   bool
 5   ct_health      2000 non-null   float64
 6   t_health       2000 non-null   float64
 7   ct_armor       2000 non-null   float64
 8   t_armor        2000 non-null   float64
 9   ct_money       2000 non-null   float64
 10  t_money        2000 non-null   float64
 11  ct_helmets     2000 non-null   float64
 12  t_helmets      2000 non-null   float64
 13  round_winner   2000 non-null   object
dtypes: bool(1), float64(11), object(2)
memory usage: 205.2+ KB

Missing Values:
time_left       0
ct_score        0
t_score         0
map             0
bomb_planted    0
ct_health       0
t_health        0
ct_armor        0
t_armor         0
ct_money        0
t_money         0
ct_helmets      0
t_helmets       0
round_winner    0
dtype: int64

Jumlah data duplikat: 39
```

```
Distribusi round_winner:
round_winner
T     1176
CT     824
Name: count, dtype: int64
```



Hasil pengecekan menunjukkan terdapat 39 data duplikat. Data duplikat perlu dihapus pada tahap preprocessing agar tidak memengaruhi proses pelatihan dan evaluasi model.

Distribusi variabel target round_winner menunjukkan bahwa kelas T sebanyak 1.176 data (58,8%), sedangkan kelas CT sebanyak 824 data (41,2%). Grafik distribusi memperlihatkan bahwa jumlah data pemenang dari tim T lebih banyak dibandingkan tim CT.

Meskipun terdapat perbedaan jumlah data pada kedua kelas, distribusinya masih cukup berimbang dan tidak menunjukkan ketimpangan kelas yang terlalu besar. Hasil EDA ini digunakan sebagai dasar untuk menentukan tahapan preprocessing sebelum model Naive Bayes dan KNN diterapkan.

3.3 Preprocessing Data

3. Preprocessing data

```
[3]: # Menghapus data duplikat
df = df.drop_duplicates()

# Menghapus baris yang memiliki missing values
df = df.dropna()

# Memisahkan fitur dan target
X = df.drop('round_winner', axis=1)
y = df['round_winner']

# Mengubah fitur kategorikal menjadi numerik
X = pd.get_dummies(X, drop_first=False)

# Membagi dataset menjadi data training dan testing
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Normalisasi fitur menggunakan StandardScaler
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Jumlah data training:", X_train.shape[0])
print("Jumlah data testing:", X_test.shape[0])
print("Jumlah fitur setelah encoding:", X_train.shape[1])

# TAQIYA RIZKIPUDENTA_202431180
```

```
Jumlah data training: 1568
Jumlah data testing: 393
Jumlah fitur setelah encoding: 19
```

Pada tahap preprocessing, dilakukan penghapusan data duplikat menggunakan fungsi drop_duplicates() dan penghapusan data yang memiliki nilai kosong menggunakan dropna(). Selanjutnya, dataset dipisahkan menjadi fitur (X) dan target (y), dengan round_winner sebagai variabel target.

Atribut kategorikal pada fitur diubah menjadi numerik menggunakan pd.get_dummies(). Dataset kemudian dibagi menjadi data training dan testing dengan rasio 80:20 serta menggunakan stratify=y untuk mempertahankan proporsi kelas target.

Berdasarkan output, diperoleh 1.568 data training dan 393 data testing, dengan total 19 fitur setelah proses encoding. Jumlah data tersebut merupakan hasil pembagian dari dataset setelah data duplikat dihapus.

Tahap normalisasi dilakukan menggunakan StandardScaler() untuk menyamakan skala nilai pada setiap fitur. Proses ini membantu algoritma KNN dalam menghitung jarak antar-data secara lebih proporsional. Hasil preprocessing kemudian digunakan sebagai input dalam proses pelatihan dan pengujian model Naive Bayes serta KNN.

3.4 Implementasi Algoritma Naive Bayes

4. Implementasi algoritma Naive Bayes

```
[4]: # Membuat model Naive Bayes
model_nb = GaussianNB()

# Melatih model
model_nb.fit(X_train_scaled, y_train)

# Melakukan prediksi
y_pred_nb = model_nb.predict(X_test_scaled)

# Menampilkan hasil prediksi
print("Hasil Prediksi Naive Bayes:")
print(y_pred_nb)

# TAQIYA RIZKIPUDENTA_202431180
```

```
Hasil Prediksi Naive Bayes:
['CT' 'T' 'T' 'T' 'T' 'T' 'CT' 'CT' 'T' 'T' 'T' 'CT' 'T' 'CT' 'T' 'CT'
 'CT' 'CT' 'T' 'CT' 'CT' 'T' 'T' 'T' 'T' 'CT' 'CT' 'CT' 'CT' 'T' 'CT' 'T'
 'CT' 'T' 'CT' 'T' 'T' 'CT' 'T' 'CT' 'CT' 'T' 'CT' 'T' 'T' 'T' 'CT' 'T'
 'T' 'T' 'CT' 'CT' 'T' 'T' 'CT' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'CT' 'T'
 'T' 'CT' 'T' 'CT' 'CT' 'T' 'CT' 'T' 'T' 'T' 'T' 'T' 'CT' 'CT' 'CT' 'CT'
 'CT' 'T' 'CT' 'CT' 'T' 'T' 'CT' 'T' 'T' 'T' 'T' 'CT' 'CT' 'T' 'CT' 'T'
 'T' 'T' 'T' 'CT' 'CT' 'T' 'T' 'T' 'CT' 'T' 'T' 'CT' 'T' 'T' 'CT' 'T' 'T'
 'T' 'T' 'CT' 'CT' 'T' 'T' 'T' 'CT' 'T' 'T' 'CT' 'T' 'CT' 'CT' 'CT' 'CT'
 'T' 'CT' 'CT' 'T' 'CT' 'T' 'T' 'T' 'CT' 'T' 'T' 'T' 'CT' 'CT' 'T' 'CT'
 'T' 'CT' 'CT' 'T' 'CT' 'T' 'T' 'CT' 'T' 'CT' 'T' 'CT' 'T' 'T' 'T' 'CT'
 'T' 'T' 'T' 'CT' 'T' 'T' 'CT' 'T' 'T' 'T' 'CT' 'T' 'CT' 'CT' 'T' 'T' 'T'
 'T' 'T' 'CT' 'CT' 'T' 'CT' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'CT' 'T'
 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'CT' 'T' 'T' 'CT' 'T' 'CT' 'CT' 'T' 'T' 'T'
 'CT' 'CT' 'T' 'T' 'CT' 'T' 'T' 'CT' 'T' 'CT' 'T' 'T' 'T' 'T' 'CT' 'T' 'T'
 'T' 'T' 'CT' 'CT' 'CT' 'T' 'CT' 'CT' 'T' 'CT' 'CT' 'T' 'CT' 'T' 'T' 'T'
 'T' 'CT' 'T' 'T' 'T' 'CT' 'T' 'T' 'T' 'T' 'CT' 'T' 'T' 'T' 'T' 'CT' 'T' 'T'
 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'CT' 'T' 'T' 'CT' 'T' 'CT' 'T' 'T' 'T' 'T'
 'CT' 'CT' 'T' 'T' 'CT' 'T' 'T' 'CT' 'T' 'CT' 'T' 'T' 'T' 'T' 'CT' 'T' 'T'
 'T' 'T' 'CT' 'CT' 'CT' 'T' 'CT' 'CT' 'T' 'CT' 'CT' 'T' 'CT' 'T' 'T' 'T'
 'T' 'CT' 'T' 'T' 'T' 'CT' 'T' 'T' 'T' 'T' 'CT' 'T' 'T' 'T' 'T' 'CT' 'T' 'T'
 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T'
 'T' 'T' 'T' 'T' 'CT' 'T' 'CT' 'T' 'T' 'T' 'T' 'CT' 'CT' 'T' 'T' 'T' 'T' 'CT'
 'T' 'T' 'T' 'CT' 'T' 'T' 'T' 'T' 'T' 'CT' 'T' 'T' 'T' 'CT' 'T' 'CT' 'CT'
 'T' 'T' 'CT' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'CT' 'T' 'CT']
```

Pada tahap ini, algoritma Gaussian Naive Bayes diterapkan untuk melakukan klasifikasi terhadap variabel target round_winner. Model dibuat menggunakan GaussianNB() dan dilatih menggunakan data training melalui fungsi fit().

Setelah proses training selesai, model melakukan prediksi terhadap data testing menggunakan fungsi predict(). Hasil prediksi disimpan dalam variabel y_pred_nb.

Berdasarkan output, model menghasilkan prediksi kelas pemenang ronde berupa T (Terrorist) dan CT (Counter-Terrorist) untuk setiap data testing. Hasil tersebut menunjukkan bahwa model Naive Bayes telah berhasil melakukan proses klasifikasi.

Namun, output prediksi ini belum menunjukkan tingkat ketepatan model. Evaluasi lebih lanjut menggunakan accuracy, precision, recall, F1-score, dan confusion matrix diperlukan untuk mengetahui seberapa baik model memprediksi pemenang ronde.

3.5 Implementasi Algoritma K-Nearest Neighbor (KNN)

5. Implementasi algoritma K-Nearest Neighbor (KNN)

[5]:
```
# Membuat model KNN dengan k = 5
model_knn = KNeighborsClassifier(n_neighbors=5)

# Melatih model
model_knn.fit(X_train_scaled, y_train)

# Melakukan prediksi
y_pred_knn = model_knn.predict(X_test_scaled)

# Menampilkan hasil prediksi
print("Hasil Prediksi KNN:")
print(y_pred_knn)

# TAQIYA RIZKIPUDENTA_202431180
```

```
Hasil Prediksi KNN:
['CT' 'CT' 'T' 'T' 'T' 'CT' 'CT' 'T' 'T' 'T' 'T' 'T' 'CT' 'CT' 'T' 'T' 'T'
 'CT' 'T' 'T' 'CT' 'T' 'T' 'T' 'T' 'T' 'CT' 'CT' 'CT' 'T' 'CT' 'CT' 'CT'
 'T' 'T' 'T' 'T' 'CT' 'CT' 'T' 'CT' 'T' 'CT' 'CT' 'T' 'T' 'CT' 'T' 'T'
 'CT' 'CT' 'T' 'T' 'T' 'CT' 'T' 'T' 'T' 'T' 'T' 'CT' 'CT' 'T' 'CT' 'T' 'T'
 'T' 'T' 'CT' 'CT' 'T' 'T' 'T' 'T' 'T' 'T' 'CT' 'CT' 'T' 'CT' 'T' 'T' 'T'
 'T' 'CT' 'CT' 'T' 'CT' 'T' 'CT' 'T' 'CT' 'CT' 'T' 'T' 'T' 'CT' 'T' 'T'
 'T' 'T' 'CT' 'T' 'CT' 'CT' 'CT' 'T' 'T' 'CT' 'T' 'T' 'CT' 'CT' 'T' 'T'
 'CT' 'CT' 'T' 'T' 'T' 'T' 'CT' 'T' 'T' 'T' 'T' 'CT' 'T' 'CT' 'T' 'T' 'T'
 'T' 'T' 'CT' 'CT' 'CT' 'T' 'T' 'T' 'T' 'T' 'CT' 'T' 'T' 'CT' 'T' 'CT'
 'CT' 'T' 'CT' 'CT' 'T' 'CT' 'T' 'CT' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'CT'
 'CT' 'CT' 'T' 'CT' 'T' 'T' 'T' 'CT' 'CT' 'CT' 'CT' 'T' 'T' 'T' 'T' 'T'
 'CT' 'CT' 'T' 'T' 'CT' 'T' 'CT' 'T' 'CT' 'T' 'CT' 'T' 'T' 'CT' 'T' 'CT'
 'T' 'CT' 'T' 'T' 'T' 'T' 'CT' 'T' 'T' 'CT' 'T' 'CT' 'CT' 'T' 'CT' 'T' 'T'
 'CT' 'T' 'CT' 'CT' 'T' 'T' 'T' 'T' 'CT' 'T' 'T' 'T' 'T' 'CT' 'CT' 'T' 'T'
 'T' 'CT' 'T' 'T' 'T' 'CT' 'CT' 'CT' 'CT' 'CT' 'CT' 'T' 'T' 'T' 'CT' 'T'
 'CT' 'T' 'T' 'T' 'T' 'CT' 'T' 'T' 'CT' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T'
 'T' 'T' 'CT' 'T' 'CT' 'T' 'T' 'T' 'T' 'T' 'CT' 'T' 'CT' 'CT' 'CT' 'T' 'T'
 'CT' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'CT' 'CT' 'T' 'T' 'T' 'CT' 'CT' 'T' 'CT'
 'CT' 'CT' 'T' 'CT' 'CT' 'CT' 'CT' 'T' 'T' 'T' 'T' 'CT' 'T' 'CT' 'T' 'CT' 'CT'
 'T' 'CT' 'CT' 'T' 'T' 'T' 'T' 'T' 'T' 'CT' 'T' 'CT' 'T' 'T' 'T' 'T' 'CT'
 'T' 'T' 'CT' 'CT' 'CT' 'T' 'CT' 'CT' 'CT' 'CT' 'T' 'T' 'CT' 'T' 'T' 'T'
 'T' 'T' 'CT' 'CT' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'T' 'CT' 'CT' 'CT' 'CT' 'T'
 'CT' 'T' 'CT' 'T' 'T' 'T' 'T' 'T' 'CT' 'T' 'T' 'T' 'CT' 'T' 'CT' 'CT' 'T'
 'T' 'CT' 'CT' 'T' 'T' 'T' 'T' 'T' 'T' 'CT' 'T' 'T']
```

Pada tahap ini, algoritma K-Nearest Neighbor digunakan untuk melakukan klasifikasi terhadap variabel target round_winner. Model dibuat menggunakan KNeighborsClassifier(n_neighbors=5), yang berarti proses klasifikasi menggunakan lima tetangga terdekat untuk menentukan kelas prediksi.

Model dilatih menggunakan data training melalui fungsi fit(), kemudian digunakan untuk memprediksi data testing menggunakan fungsi predict(). Hasil prediksi disimpan dalam variabel y_pred_knn.

Berdasarkan output, model KNN menghasilkan prediksi kelas pemenang ronde berupa T (Terrorist) dan CT (Counter-Terrorist) pada data testing. Hasil tersebut menunjukkan bahwa model berhasil melakukan klasifikasi berdasarkan kedekatan karakteristik data.

Output prediksi ini selanjutnya digunakan dalam tahap evaluasi untuk mengetahui tingkat ketepatan model KNN dan membandingkan kinerjanya dengan algoritma Naive Bayes.

3.6 Evaluasi Model & Visualisasi Perbandingan Model

6. Evaluasi model

```
[6]: # Fungsi evaluasi model
def evaluasi_model(y_test, y_pred, nama_model):
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(
        y_test, y_pred,
        average='weighted',
        zero_division=0
    )
    recall = recall_score(
        y_test, y_pred,
        average='weighted',
        zero_division=0
    )
    f1 = f1_score(
        y_test, y_pred,
        average='weighted',
        zero_division=0
    )

    return {
        'Model': nama_model,
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall,
        'F1-Score': f1
    }

# Evaluasi Naive Bayes
hasil_nb = evaluasi_model(
    y_test,
    y_pred_nb,
    'Naive Bayes'
)

# Evaluasi KNN
hasil_knn = evaluasi_model(
    y_test,
    y_pred_knn,
    'KNN'
)

# Membuat tabel perbandingan
hasil_evaluasi = pd.DataFrame([
    hasil_nb,
    hasil_knn
])

print(hasil_evaluasi)

# TAQIYA RIZKIPUDENTA_202431180
```

```
         Model  Accuracy  Precision    Recall  F1-Score
0  Naive Bayes  0.737913   0.735363  0.737913  0.734851
1          KNN  0.798982   0.797821  0.798982  0.797755
```

7. Visualisasi perbandingan model

```
[7]: # Mengambil nilai accuracy dari hasil evaluasi
akurasi = hasil_evaluasi['Accuracy']

# Membuat grafik perbandingan
plt.figure(figsize=(8, 5))

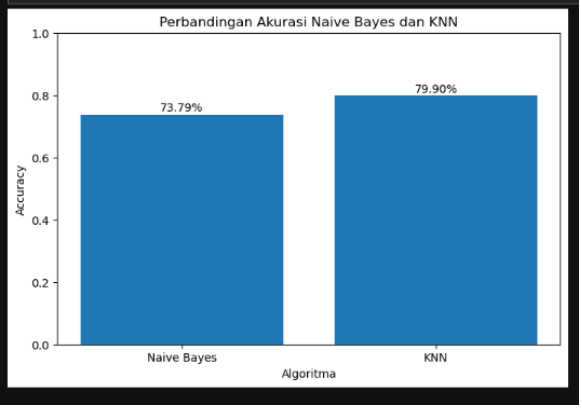
plt.bar(
    hasil_evaluasi['Model'],
    akurasi
)

plt.title('Perbandingan Akurasi Naive Bayes dan KNN')
plt.xlabel('Algoritma')
plt.ylabel('Accuracy')
plt.ylim(0, 1)

for i, nilai in enumerate(akurasi):
    plt.text(i, nilai + 0.01, f'{nilai:.2%}', ha='center')

plt.show()

# TAQIYA RIZKIPUDENTA_202431180
```



Pada tahap evaluasi, dilakukan pengukuran kinerja algoritma Naive Bayes dan KNN menggunakan metrik accuracy, precision, recall, dan F1-score. Hasil evaluasi ditampilkan dalam bentuk tabel untuk membandingkan performa kedua algoritma pada data testing.

Berdasarkan output, diperoleh hasil evaluasi sebagai berikut:

Metrik	Naive Bayes	KNN
Accuracy	73,79%	79,90%
Precision	73,54%	79,78%
Recall	73,79%	79,90%
F1-Score	73,49%	79,78%

Hasil tersebut menunjukkan bahwa algoritma KNN memiliki nilai accuracy sebesar 79,90%, lebih tinggi dibandingkan Naive Bayes yang memperoleh 73,79%. KNN juga menghasilkan nilai precision dan F1-score yang lebih tinggi.

Dengan demikian, berdasarkan hasil pengujian pada dataset ini, KNN menunjukkan performa klasifikasi yang lebih baik dibandingkan Naive Bayes. KNN mampu menghasilkan prediksi yang lebih sesuai dengan kelas aktual pada data testing.

7. Visualisasi Perbandingan Model

Visualisasi perbandingan akurasi dibuat menggunakan grafik batang untuk memperlihatkan perbedaan kinerja kedua algoritma. Nilai accuracy diambil dari tabel hasil evaluasi model.

Berdasarkan grafik, algoritma Naive Bayes memperoleh akurasi sebesar 73,79%, sedangkan algoritma KNN memperoleh akurasi sebesar 79,90%. Terdapat selisih akurasi sebesar 6,11 poin persentase antara kedua algoritma.

Grafik menunjukkan bahwa KNN memiliki nilai akurasi yang lebih tinggi dibandingkan Naive Bayes. Hal ini menunjukkan bahwa pada pembagian data dan parameter yang digunakan dalam praktikum, KNN memberikan hasil klasifikasi yang lebih baik dalam memprediksi pemenang ronde Counter-Strike 2.

3.7 Confusion Matrix

8. Confusion matrix

[8]:
```
# Confusion Matrix Naive Bayes
cm_nb = confusion_matrix(
    y_test,
    y_pred_nb,
    labels=['T', 'CT']
)

plt.figure(figsize=(6, 4))
sns.heatmap(
    cm_nb,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['T', 'CT'],
    yticklabels=['T', 'CT']
)

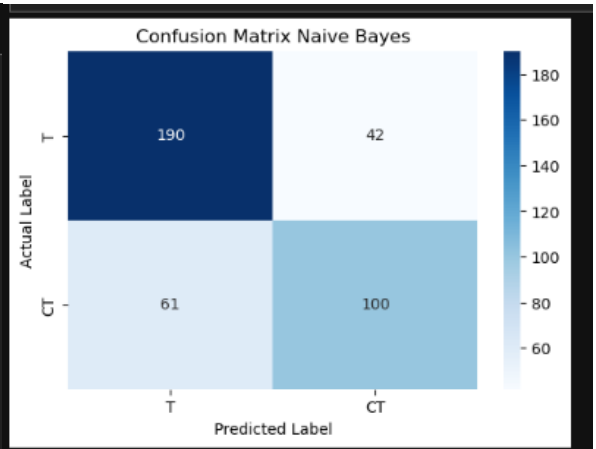
plt.title('Confusion Matrix Naive Bayes')
plt.xlabel('Predicted Label')
plt.ylabel('Actual Label')
plt.show()

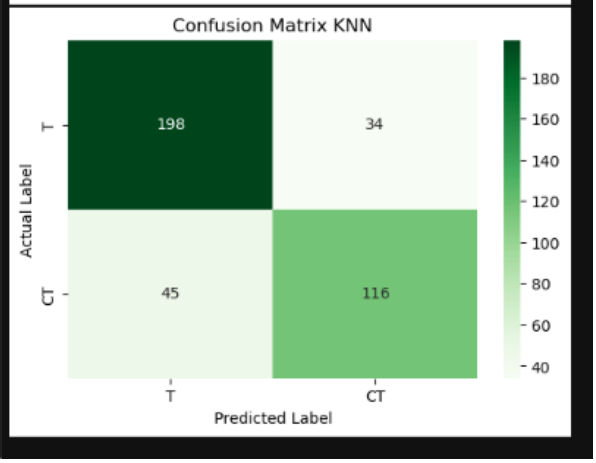

# Confusion Matrix KNN
cm_knn = confusion_matrix(
    y_test,
    y_pred_knn,
    labels=['T', 'CT']
)

plt.figure(figsize=(6, 4))
sns.heatmap(
    cm_knn,
    annot=True,
    fmt='d',
    cmap='Greens',
    xticklabels=['T', 'CT'],
    yticklabels=['T', 'CT']
)

plt.title('Confusion Matrix KNN')
plt.xlabel('Predicted Label')
plt.ylabel('Actual Label')
plt.show()

# TAQIYA RIZKIPUDENTA_202431180
```





Confusion matrix digunakan untuk melihat perbandingan antara hasil prediksi model dengan kelas aktual pada data testing. Matriks ini menunjukkan jumlah prediksi yang benar maupun salah untuk kelas T (Terrorist) dan CT (Counter-Terrorist).

Berdasarkan confusion matrix Naive Bayes, model berhasil memprediksi 190 data kelas T dan 100 data kelas CT dengan benar. Sementara itu, terdapat 42 data kelas T yang diprediksi sebagai CT dan 61 data kelas CT yang diprediksi sebagai T.

Pada confusion matrix KNN, model berhasil memprediksi 198 data kelas T dan 116 data kelas CT dengan benar. Kesalahan prediksi terdiri dari 34 data kelas T yang diprediksi sebagai CT serta 45 data kelas CT yang diprediksi sebagai T.

Hasil tersebut menunjukkan bahwa KNN memiliki jumlah prediksi benar yang lebih tinggi dibandingkan Naive Bayes, yaitu sebanyak 314 data, sedangkan Naive Bayes menghasilkan 290 prediksi benar. KNN juga memiliki jumlah kesalahan klasifikasi yang lebih sedikit.

Dengan demikian, hasil confusion matrix mendukung hasil evaluasi sebelumnya bahwa algoritma KNN memiliki kinerja klasifikasi yang lebih baik dibandingkan Naive Bayes pada dataset Counter-Strike 2 yang digunakan dalam praktikum.

3.8 Menampilkan Hasil Prediksi Aktual dan Model

9. Menampilkan hasil prediksi aktual dan model

[9]:
```
# Membuat tabel hasil prediksi
hasil_prediksi = pd.DataFrame({
    'Aktual': y_test.values,
    'Prediksi Naive Bayes': y_pred_nb,
    'Prediksi KNN': y_pred_knn
})

# Menampilkan 20 data pertama
hasil_prediksi.head(20)

# TAQIYA RIZKIPUDENTA_202431180
```

[9]:

	Aktual	Prediksi Naive Bayes	Prediksi KNN
0	CT	CT	CT
1	CT	T	CT
2	T	T	T
3	T	T	T
4	T	T	T
5	CT	T	CT
6	CT	CT	CT
7	T	CT	T
8	T	T	T
9	T	T	T
10	T	T	T
11	T	CT	T
12	CT	T	CT
13	CT	CT	CT
14	T	T	T
15	CT	CT	T
16	T	CT	T
17	CT	CT	CT
18	T	T	T
19	CT	CT	T

Bagian ini menampilkan perbandingan langsung antara kelas aktual dengan hasil prediksi dari masing-masing model (Naive Bayes dan KNN) untuk 20 data pertama pada data testing.

Berdasarkan tabel hasil prediksi di atas:

- Kolom **Aktual** menunjukkan label atau kelas yang sebenarnya dari data uji, sedangkan kolom **Prediksi Naive Bayes** dan **Prediksi KNN** menunjukkan hasil tebakan dari masing-masing algoritma.

- Melalui tampilan 20 data pertama ini, kita dapat melihat secara mendetail bagaimana performa kedua model dalam mengenali pola pada dataset Counter-Strike 2. Sebagai contoh, pada indeks ke-0, baik Naive Bayes maupun KNN berhasil memprediksi kelas CT dengan benar sesuai dengan nilai aktualnya. Namun, pada beberapa baris lain seperti indeks ke-1 dan ke-5, terlihat adanya perbedaan di mana salah satu atau kedua model menghasilkan prediksi yang keliru (misalnya nilai aktual CT tetapi diprediksi T oleh Naive Bayes).

3.9 Kesimpulan Hasil

Berdasarkan seluruh tahapan praktikum yang telah dilakukan pada dataset Counter-Strike 2, dapat ditarik beberapa kesimpulan sebagai berikut:

- **Tahapan Pengolahan Data:** Dataset awal memiliki 2.000 baris data dan 14 atribut dengan variabel target round_winner (Terrorist / T dan Counter-Terrorist / CT). Melalui tahapan *Exploratory Data Analysis* (EDA) dan *preprocessing*—yang meliputi penghapusan data duplikat, penanganan nilai kosong, *one-hot encoding* untuk fitur kategorikal, pembagian data dengan rasio 80:20 (stratifikasi), serta normalisasi fitur menggunakan StandardScaler()—diperoleh sebanyak 1.568 data *training* dan 393 data *testing* dengan 19 fitur siap pakai.

- **Performa Model Klasifikasi:** Penerapan algoritma Naive Bayes dan *K-Nearest Neighbor* (KNN dengan $k = 5$) berhasil dilakukan untuk memprediksi pemenang ronde. Berdasarkan metrik evaluasi, algoritma KNN terbukti menghasilkan kinerja yang lebih unggul dibandingkan Naive Bayes dengan tingkat **akurasi sebesar 79,90%** (*precision* 79,78%, *recall* 79,90%, dan *F1-score* 79,78%), sementara Naive Bayes memperoleh akurasi sebesar 73,79% (*precision* 73,54%, *recall* 73,79%, dan *F1-score* 73,49%).

- **Analisis Confusion Matrix & Prediksi:** Hasil *confusion matrix* memperkuat temuan tersebut, di mana KNN menghasilkan total 314 prediksi benar dengan kesalahan klasifikasi yang lebih sedikit, sedangkan Naive Bayes menghasilkan 290 prediksi benar. Tampilan perbandingan 20 data pertama juga memberikan gambaran nyata mengenai akurasi tebakan model terhadap kelas aktual pada data *testing*.

Secara keseluruhan, algoritma K-Nearest Neighbor (KNN) menunjukkan performa klasifikasi yang lebih baik dan konsisten dibandingkan Naive Bayes dalam mengenali pola serta memprediksi pemenang pada dataset Counter-Strike 2 dalam praktikum ini.

BAB IV

PENUTUP

Berdasarkan landasan teori dan seluruh rangkaian hasil praktikum yang telah dilaksanakan, dapat ditarik kesimpulan sebagai berikut:

1. **Konsep Dasar Machine Learning & Klasifikasi:** *Machine learning* merupakan cabang kecerdasan buatan yang memungkinkan komputer belajar dari data, di mana tugas klasifikasi berfungsi memetakan data ke dalam kelas diskrit (pada praktikum ini yaitu tim Terrorist / T dan Counter-Terrorist / CT).

2. **Karakteristik Dataset & Analisis Eksploratif (EDA):** Dataset awal Counter-Strike 2 memiliki 2.000 baris dan 14 atribut dengan variabel target round_winner, di mana hasil EDA menunjukkan tidak ada *missing values* namun terdapat 39 data duplikat serta proporsi kelas T sebesar 58,8% dan CT sebesar 41,2%.

3. **Prosedur Preprocessing Data:** Tahap penyiapan data berhasil membersihkan data duplikat, merubah atribut kategorikal menjadi numerik (*one-hot encoding*), serta menyamakan skala fitur menggunakan StandardScaler() untuk mengoptimalkan kinerja perhitungan algoritma.

4. **Pembagian Dataset (Train-Test Split):** Pembagian data dilakukan dengan rasio 80:20 secara terstratifikasi (stratify=y), sehingga menghasilkan 1.568 data *training* dan 393 data *testing* dengan 19 fitur siap latih.

5. **Implementasi Model Naive Bayes & KNN:** Pemodelan berhasil menerapkan algoritma Gaussian Naive Bayes dan K-Nearest Neighbors ($k = 5$) untuk melakukan klasifikasi prediksi pemenang ronde berdasarkan data pengujian.

6. **Evaluasi Kinerja Model Klasifikasi:** Berdasarkan metrik evaluasi kuantitatif, algoritma KNN terbukti lebih unggul dibanding Naive Bayes dengan akurasi 79,90% (*precision* 79,78%, *recall* 79,90%, *F1-score* 79,78%), sedangkan Naive Bayes memperoleh akurasi 73,79% (*precision* 73,54%, *recall* 73,79%, *F1-score* 73,49%).

7. **Visualisasi Perbandingan Model:** Grafik batang perbandingan nilai akurasi secara visual memperlihatkan keunggulan performa KNN dengan selisih 6,11 poin persentase lebih tinggi dibandingkan Naive Bayes.

8. **Analisis Confusion Matrix:** Matriks konfirmasi menunjukkan algoritma KNN berhasil memprediksi 314 data uji dengan benar (kesalahan 79 data), sedangkan Naive Bayes memprediksi 290 data uji dengan benar (kesalahan 103 data).

9. **Tampilan Hasil Prediksi Detail:** Tampilan perbandingan 20 data pertama secara langsung memberikan konfirmasi intuitif atas ketepatan tebakan model KNN dan Naive Bayes terhadap label asli data *testing*.

DAFTAR PUSTAKA

Annisa, C., Afdal, M., & Ahsyar, T. K. (2023). Perbandingan algoritma Naïve Bayes Classifier dan K-Nearest Neighbor pada sentimen review aplikasi Mobile JKN. *Jurnal Media Informatika Budidarma*, 7(3), 1033–1040. https://doi.org/10.30865/mib.v7i3.6242

Krisna, D. A. N., & Salamah, U. (2022). Perbandingan algoritma Naïve Bayes dan K-Nearest Neighbor untuk klasifikasi berita hoax kesehatan di media sosial Twitter. *JTIK (Jurnal Teknik Informatika Kaputama)*, 6(2), 836–845. https://doi.org/10.59697/jtik.v6i2.295

Loka, S. K. P., & Marsal, A. (2023). Perbandingan algoritma K-Nearest Neighbor dan Naïve Bayes Classifier untuk klasifikasi status gizi pada balita. MALCOM: *Indonesian Journal of Machine Learning and Computer Science*, 3(1), 8–14. https://doi.org/10.57152/malcom.v3i1.474

Prayogo, A., Fauziah, F., & Winarsih, W. (2023). Perbandingan algoritma Naïve Bayes dan K-Nearest Neighbor pada klasifikasi judul artikel pada jurnal ilmiah. *JIPI (Jurnal Ilmiah Penelitian dan Pembelajaran Informatika),* 8(4). https://jurnal.stkippgritulungagung.ac.id/index.php/jipi/article/view/4141

Sholekhah, F., Putri, A. D., Rahmaddeni, R., & Efrizoni, L. (2024). Perbandingan algoritma Naïve Bayes dan K-Nearest Neighbors untuk klasifikasi metabolik sindrom. *MALCOM: Indonesian Journal of Machine Learning and Computer Science,* 4(2), 507–514. https://doi.org/10.57152/malcom.v4i2.1249